

**LAPORAN WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)**

SEMESTER III



BKPM: ACARA 9

**NAMA:
SHERYN FEBRYLIA HENDRITA
(E41250604)**

**PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026**

1. PENDAHULUAN

Praktikum Acara 9 bertujuan untuk memahami penerapan konsep Object-Oriented Programming (OOP) pada pengembangan aplikasi web dengan pola MVC.

Pada praktikum ini dipelajari konsep Object Composition, Dependency Injection, serta Getter dan Setter. Konsep tersebut digunakan untuk membuat struktur program yang lebih terorganisasi, mudah dikembangkan, dan lebih mudah diuji.

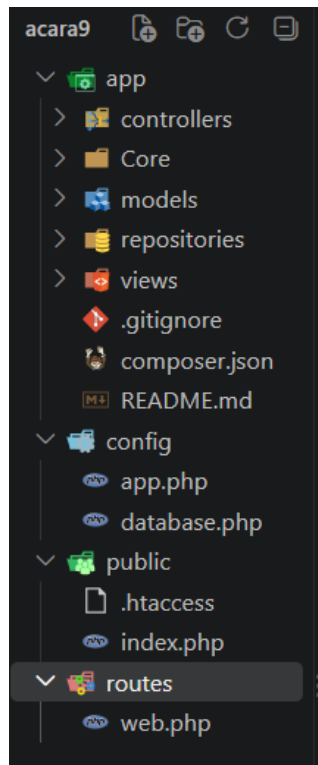
Selain itu, praktikum bertujuan agar proses akses database tidak dilakukan secara langsung oleh Controller, tetapi menggunakan Repository yang menerima dependency melalui constructor. Atribut pada class Mahasiswa juga dibuat lebih aman dengan menggunakan access modifier private serta diakses melalui getter dan setter.

2. HASIL PRAKTIKUM

1. Persiapan Project

Praktikum menggunakan project **Sistem Informasi Akademik** yang telah dibuat pada praktikum sebelumnya. Project digunakan kembali untuk menerapkan konsep OOP MVC lanjutan.

Struktur aplikasi terdiri dari beberapa bagian utama, yaitu:



2. Class Database

digunakan untuk mengelola koneksi aplikasi dengan database MySQL menggunakan PDO.

Pada class tersebut terdapat atribut:

```
private PDO $pdo;
```

Koneksi database dibuat menggunakan konfigurasi yang terdapat pada config/database.php.

Bagian penting dari class Database adalah:

```
$this->pdo = new PDO(  
    $dsn,  
    $config['username'],  
    $config['password'],  
    [  
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,  
        PDO::ATTR_DEFAULT_FETCH_MODE =>  
PDO::FETCH_ASSOC,  
        PDO::ATTR_EMULATE_PREPARES => false,
```

]

);

Class Database juga memiliki method getConnection() untuk mendapatkan object PDO.

Selain itu, terdapat method getInstance() yang menyediakan koneksi melalui static instance.

Gambar 2. Implementasi class Database

```
Database.php
app > Core > Database.php > Database > getConnection()
3  class Database
9  public function __construct()
24  };
25  }
26
27  public function getConnection(): PDO
28  {
29      return $this->pdo;
30  }
31
32  public static function getInstance(): PDO
33  {
34      if (self::$instance === null) {
35          $config = require __DIR__ . '/../../config/database.php';
36          $dsn = "mysql:host={$config['host']};dbname={$config['dbname']};charset={$config['charset']}";
37          self::$instance = new PDO(
38              $dsn,
39              $config['username'],
40              $config['password'],
41              [
42                  PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
43                  PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
44                  PDO::ATTR_EMULATE_PREPARES => false,
45              ]
46          );
47      }
48      return self::$instance;
49  }
50  }
51  }
52  }
53  }
54  }
```

3. Class Mahasiswa dengan Atribut Private

Pada Acara 9, atribut pada class Mahasiswa diubah menjadi private.

Atribut yang digunakan adalah:

private int \$id;

private string \$nim;

private string \$nama;

private string \$email;

private int \$prodi_id;

private int \$angkatan;

Penggunaan private bertujuan untuk membatasi akses langsung terhadap data yang terdapat di dalam object.

Data tersebut kemudian diakses dan diubah melalui getter dan setter.

Gambar 3. Atribut private pada class Mahasiswa

```
app > models > Mahasiswa.php > Mahasiswa
1  <?php
2
3  0 references | 0 implementations
4  class Mahasiswa
5  {
6      2 references
7      private int $id;
8      2 references
9      private string $nim;
10     2 references
11     private string $nama;
12     2 references
13     private string $email;
14     2 references
15     private int $prodi_id;
16     2 references
17     private int $angkatan;
18
19     0 references | 0 overrides
20     public function getId(): int
21     {
22         return $this->id;
23     }
24
25     0 references | 0 overrides
26     public function setId(int $id): void
27     {
28         $this->id = $id;
29     }
30
31     0 references | 0 overrides
32     public function getNim(): string
33     {
34         return $this->nim;
35     }
36
37     0 references | 0 overrides
38     public function setNim(string $nim): void
39
40     app > models > Mahasiswa.php > Mahasiswa > getId()
41     class Mahasiswa
42
43     0 references | 0 overrides
44     public function setNim(string $nim): void
45     {
46         if (!is_numeric($nim)) {
47             throw new InvalidArgumentException('NIM harus berupa angka.');
```

```

app > models > Mahasiswa.php > Mahasiswa > getId()
3  class Mahasiswa
49
50      0 references | 0 overrides
51      public function getEmail(): string
52      {
53          return $this->email;
54      }
55
56      0 references | 0 overrides
57      public function setEmail(string $email): void
58      {
59          $this->email = $email;
60      }
61
62      0 references | 0 overrides
63      public function getProdiId(): int
64      {
65          return $this->prodi_id;
66      }
67
68      0 references | 0 overrides
69      public function setProdiId(int $prodi_id): void
70      {
71          $this->prodi_id = $prodi_id;
72      }
73
74      0 references | 0 overrides
75      public function getAngkatan(): int
76      {
77          return $this->angkatan;
78      }
79
80      0 references | 0 overrides
81      public function setAngkatan(int $angkatan): void
82      {
83          $this->angkatan = $angkatan;
84      }

```

4. Penerapan Getter dan Setter

Setelah atribut dibuat private, dibuat getter untuk mengambil nilai dan setter untuk mengubah nilai atribut.

Contoh getter NIM:

```

public function getNim(): string
{
    return $this->nim;
}

```

Contoh setter NIM:

```

public function setNim(string $nim): void
{
    if (!is_numeric($nim)) {
        throw new InvalidArgumentException(
            'NIM harus berupa angka.'
        );
    }
}

```

```
}
```

```
$this->nim = $nim;
```

```
}
```

Getter digunakan untuk mengambil nilai atribut, sedangkan setter digunakan untuk memberikan nilai baru sekaligus dapat digunakan untuk melakukan validasi.

Gambar 4. Getter dan Setter pada class Mahasiswa

```
22 public function getNim(): string
23 {
24     return $this->nim;
25 }
26
27 0 references | 0 overrides
28 public function setNim(string $nim): void
29 {
30     if (!is_numeric($nim)) {
31         throw new InvalidArgumentException('NIM harus
32     }
33     $this->nim = $nim;
34 }
35
36 0 references | 0 overrides
37 public function getNama(): string
38 {
39     return $this->nama;
40 }
41
42 0 references | 0 overrides
43 public function setNama(string $nama): void
44 {
45     if (trim($nama) === '') {
46         throw new InvalidArgumentException('Nama maha
47     }
48     $this->nama = $nama;
49 }
```

5. Validasi pada Setter

Setter pada class Mahasiswa tidak hanya digunakan untuk mengubah nilai atribut, tetapi juga digunakan untuk melakukan validasi.

Pada setter NIM, dilakukan pengecekan agar NIM harus berupa angka:

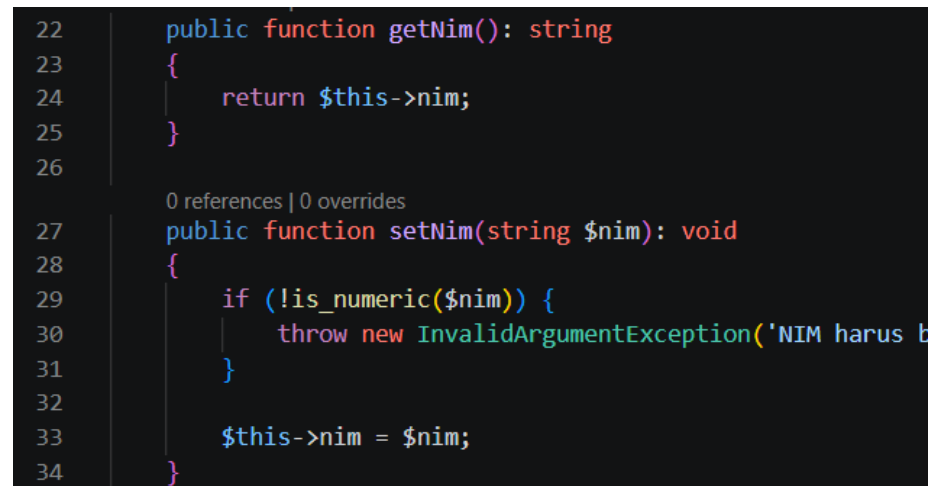
```
if (!is_numeric($nim)) {
    throw new InvalidArgumentException(
        'NIM harus berupa angka.'
    );
}
```

Pada setter nama, dilakukan pengecekan agar nama tidak boleh kosong:

```
if (trim($nama) === "") {  
    throw new InvalidArgumentException(  
        'Nama mahasiswa tidak boleh kosong.'  
    );  
}
```

Dengan adanya validasi tersebut, data yang dimasukkan ke dalam object dapat dikontrol sebelum digunakan oleh bagian aplikasi lainnya.

Gambar 5. Validasi pada Setter Mahasiswa



```
22     public function getNim(): string  
23     {  
24         return $this->nim;  
25     }  
26  
27     0 references | 0 overrides  
28     public function setNim(string $nim): void  
29     {  
30         if (!is_numeric($nim)) {  
31             throw new InvalidArgumentException('NIM harus b  
32         }  
33         $this->nim = $nim;  
34     }
```

6. MahasiswaRepository

MahasiswaRepository digunakan untuk menangani proses akses data mahasiswa ke database.

Class ini memiliki dependency berupa object Database:

```
class MahasiswaRepository  
{  
    private Database $database;  
  
    public function __construct(Database $database)  
    {  
        $this->database = $database;  
    }  
}
```

Object Database diberikan melalui constructor. Teknik ini merupakan penerapan Dependency Injection.

Repository kemudian memiliki method untuk menangani proses data mahasiswa, yaitu:

all()

find()

create()

update()

delete()

Dengan adanya Repository, proses query database tidak perlu ditulis langsung pada Controller.

Gambar 6. Dependency Injection pada MahasiswaRepository

```
app > repositories > MahasiswaRepository.php > ...
1  <?php
2
3  require_once __DIR__ . '/../Models/Mahasiswa.php';
4  class MahasiswaRepository
5  {
6      2 references
7      private Database $database;
8
9      1 reference | 0 overrides
10     public function __construct(Database $database)
11     {
12         $this->database = $database;
13     }
14
15     7 references
16     private function pdo(): PDO
17     {
18         return $this->database->getConnection();
19     }
20
21     1 reference | 0 overrides
22     public function all(string $keyword = ''): array
23     {
24         if ($keyword != '') {
25             $stmt = $this->pdo()->prepare("
26             SELECT
27             m.*,
28             p.nama AS prodi_nama
29             FROM mahasiswa m
30             JOIN prodi p ON m.prodi_id = p.id
31             WHERE m.nim LIKE :nim
32             OR m.nama LIKE :nama
33             ORDER BY m.nim
34             ");
35             $search = '%' . $keyword . '%';
```

7. Method CRUD pada MahasiswaRepository

Repository menangani operasi CRUD terhadap data mahasiswa.

Untuk mengambil seluruh data digunakan method:

```
public function all(string $keyword = ""): array
```

Method tersebut juga menyediakan pencarian berdasarkan NIM atau nama.

Untuk mencari satu data digunakan:

```
public function find(int $id): ?array
```

Untuk menambahkan data digunakan:

```
public function create(array $data): int
```

Untuk mengubah data digunakan:

```
public function update(int $id, array $data): bool
```

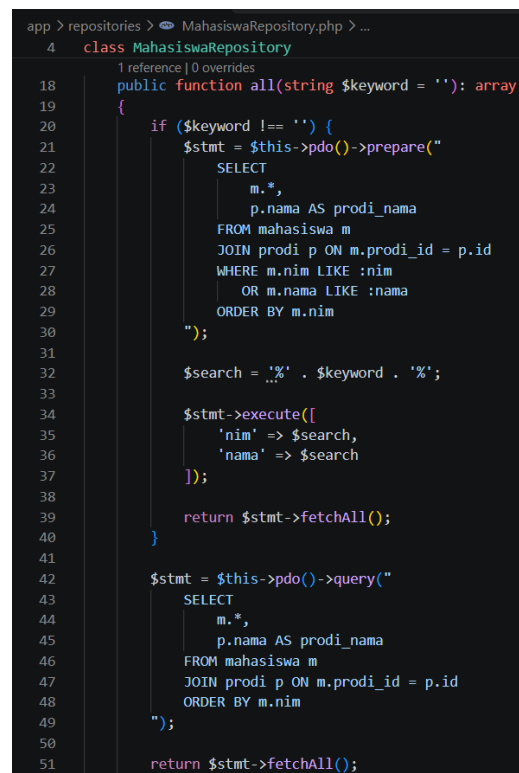
Sedangkan untuk menghapus data digunakan:

```
public function delete(int $id): bool
```

Dengan pembagian tersebut, Repository bertanggung jawab terhadap proses komunikasi antara aplikasi dengan database.

Gambar 7. Method CRUD pada MahasiswaRepository

method all()



```
app > repositories > MahasiswaRepository.php > ...
4 class MahasiswaRepository
    1 reference | 0 overrides
18 public function all(string $keyword = ''): array
19 {
20     if ($keyword !== '') {
21         $stmt = $this->pdo()->prepare("
22             SELECT
23                 m.*,
24                 p.nama AS prodi_nama
25             FROM mahasiswa m
26             JOIN prodi p ON m.prodi_id = p.id
27             WHERE m.nim LIKE :nim
28                OR m.nama LIKE :nama
29             ORDER BY m.nim
30         ");
31
32         $search = '%' . $keyword . '%';
33
34         $stmt->execute([
35             'nim' => $search,
36             'nama' => $search
37         ]);
38
39         return $stmt->fetchAll();
40     }
41
42     $stmt = $this->pdo()->query("
43         SELECT
44             m.*,
45             p.nama AS prodi_nama
46         FROM mahasiswa m
47         JOIN prodi p ON m.prodi_id = p.id
48         ORDER BY m.nim
49     ");
50
51     return $stmt->fetchAll();
52 }
```

find()

```

app > repositories > MahasiswaRepository.php > ...
4  class MahasiswaRepository
54  public function find(int $id): ?array
55  {
56      $stmt = $this->pdo()->prepare(
57          "SELECT * FROM mahasiswa WHERE id = :id"
58      );
59
60      $stmt->execute([
61          'id' => $id
62      ]);
63
64      $data = $stmt->fetch();
65
66      return $data ? : null;
67  }
68
69  1 reference | 0 overrides
70  public function create(array $data): int
71  {
72      $stmt = $this->pdo()->prepare(
73          INSERT INTO mahasiswa
74          (nim, nama, email, prodi_id, angkatan)
75          VALUES
76          (:nim, :nama, :email, :prodi_id, :angkatan)
77      );
78
79      $stmt->execute([
80          'nim' => $data['nim'],
81          'nama' => $data['nama'],
82          'email' => $data['email'],
83          'prodi_id' => $data['prodi_id'],
84          'angkatan' => $data['angkatan']
85      ]);
86
87      return (int) $this->pdo()->lastInsertId();
88  }

```

create()

```

69  public function create(array $data): int
70  {
71      $stmt = $this->pdo()->prepare(
72          INSERT INTO mahasiswa
73          (nim, nama, email, prodi_id, angkatan)
74          VALUES
75          (:nim, :nama, :email, :prodi_id, :angkatan)
76      );
77
78      $stmt->execute([
79          'nim' => $data['nim'],
80          'nama' => $data['nama'],
81          'email' => $data['email'],
82          'prodi_id' => $data['prodi_id'],
83          'angkatan' => $data['angkatan']
84      ]);
85
86      return (int) $this->pdo()->lastInsertId();
87  }
88

```

update()

```

89     public function update(int $id, array $data): bool
90     {
91         $stmt = $this->pdo()->prepare("
92             UPDATE mahasiswa
93             SET
94                 nim = :nim,
95                 nama = :nama,
96                 email = :email,
97                 prodi_id = :prodi_id,
98                 angkatan = :angkatan
99             WHERE id = :id
100         ");
101
102         return $stmt->execute([
103             'id' => $id,
104             'nim' => $data['nim'],
105             'nama' => $data['nama'],
106             'email' => $data['email'],
107             'prodi_id' => $data['prodi_id'],
108             'angkatan' => $data['angkatan']
109         ]);
110     }
111

```

dan delete()

```

112     public function delete(int $id): bool
113     {
114         $stmt = $this->pdo()->prepare(
115             "DELETE FROM mahasiswa WHERE id = :id"
116         );
117
118         return $stmt->execute([
119             'id' => $id
120         ]);
121     }
122

```

pada MahasiswaRepository.php.

8. Prepared Statement pada Repository

Pada proses pengolahan database, Repository menggunakan prepared statement.

Contohnya pada proses penambahan data:

```

$stmt = $this->pdo()->prepare("
    INSERT INTO mahasiswa
    (nim, nama, email, prodi_id, angkatan)
    VALUES

```

```
(:nim, :nama, :email, :prodi_id, :angkatan)
");
```

```
$stmt->execute([
    'nim' => $data['nim'],
    'nama' => $data['nama'],
    'email' => $data['email'],
    'prodi_id' => $data['prodi_id'],
    'angkatan' => $data['angkatan']
]);
```

Prepared statement juga digunakan pada proses pencarian, perubahan, dan penghapusan data.

Penggunaan prepared statement membuat nilai yang diberikan ke query dipisahkan dari struktur SQL.

Gambar 8. Penggunaan prepared statement pada MahasiswaRepository

```

69     public function create(array $data): int
70     {
71         $stmt = $this->pdo()->prepare("
72             INSERT INTO mahasiswa
73             (nim, nama, email, prodi_id, angkatan)
74             VALUES
75             (:nim, :nama, :email, :prodi_id, :angkatan)
76             ");
77
78         $stmt->execute([
79             'nim' => $data['nim'],
80             'nama' => $data['nama'],
81             'email' => $data['email'],
82             'prodi_id' => $data['prodi_id'],
83             'angkatan' => $data['angkatan']
84         ]);
85
86         return (int) $this->pdo()->lastInsertId();
87     }
88

```

9. MahasiswaController

MahasiswaController bertugas menangani alur permintaan yang berkaitan dengan data mahasiswa.

Pada class ini terdapat property:

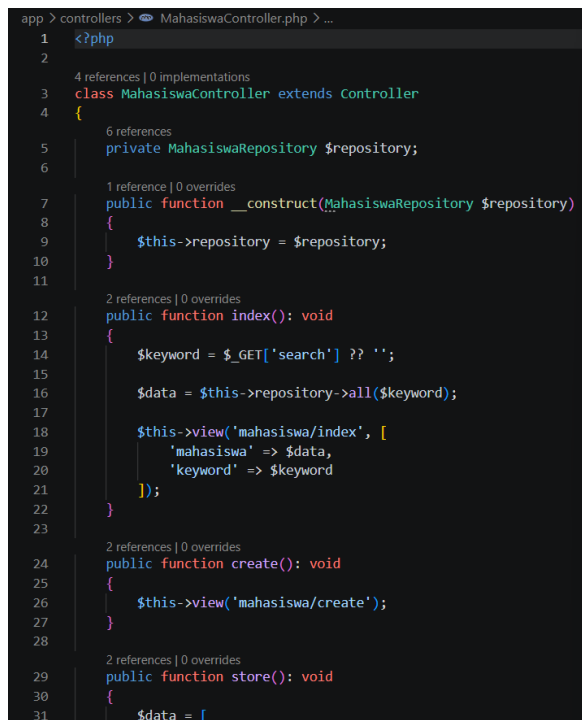
```
private MahasiswaRepository $repository;
```

Kemudian Repository diberikan melalui constructor:

```
public function __construct(MahasiswaRepository $repository)
{
    $this->repository = $repository;
}
```

Dengan demikian, Controller tidak membuat object Repository sendiri di dalam setiap method.

Gambar 9. Dependency Injection pada MahasiswaController



```
app > controllers > MahasiswaController.php > ...
1  <?php
2
3  4 references | 0 implementations
4  class MahasiswaController extends Controller
5  {
6      6 references
7      private MahasiswaRepository $repository;
8
9      1 reference | 0 overrides
10     public function __construct(MahasiswaRepository $repository)
11     {
12         $this->repository = $repository;
13     }
14
15     2 references | 0 overrides
16     public function index(): void
17     {
18         $keyword = $_GET['search'] ?? '';
19
20         $data = $this->repository->all($keyword);
21
22         $this->view('mahasiswa/index', [
23             'mahasiswa' => $data,
24             'keyword' => $keyword
25         ]);
26     }
27
28     2 references | 0 overrides
29     public function create(): void
30     {
31         $this->view('mahasiswa/create');
32     }
33
34     2 references | 0 overrides
35     public function store(): void
36     {
37         $data = [
```

```

app > controllers > MahasiswaController.php > ...
3   class MahasiswaController extends Controller
29   public function store(): void
33       'nama' => $_POST['nama'] ?? '',
34       'email' => $_POST['email'] ?? '',
35       'prodi_id' => $_POST['prodi_id'] ?? '',
36       'angkatan' => $_POST['angkatan'] ?? ''
37   ];
38
39   $this->repository->create($data);
40
41   $this->redirect('/acara9/public/index.php?route=mahasiswa')
42   }
43
1 reference | 0 overrides
44   public function edit(): void
45   {
46       $id = (int) ($_GET['id'] ?? 0);
47
48       $mahasiswa = $this->repository->find($id);
49
50       if (!$mahasiswa) {
51           die('Data mahasiswa tidak ditemukan.');
```

```

52       }
53
54       $this->view('mahasiswa/edit', [
55           'mahasiswa' => $mahasiswa
56       ]);
57   }
58
1 reference | 0 overrides
59   public function update(): void
60   {
61       $id = (int) ($_POST['id'] ?? 0);
62
63       $data = [
64           'nim' => $_POST['nim'] ?? '',
65           'nama' => $_POST['nama'] ?? '',
66           'email' => $_POST['email'] ?? '',
67           'prodi_id' => $_POST['prodi_id'] ?? '',
68           'angkatan' => $_POST['angkatan'] ?? ''
69       ];
70
71       $this->repository->update($id, $data);
72
73       $this->redirect('/acara9/public/index.php?route=mahasiswa')
74   }
75
0 references | 0 overrides
76   public function delete(): void
77   {
78       $id = (int) ($_GET['id'] ?? 0);
79
80       $this->repository->delete($id);
81
82       $this->redirect('/acara9/public/index.php?route=mahasiswa')
83   }
84 }
```

10. Menampilkan Data Mahasiswa

Method `index()` pada Controller digunakan untuk menampilkan data mahasiswa.

Kode yang digunakan:

```

public function index(): void
{
    $keyword = $_GET['search'] ?? '';

    $data = $this->repository->all($keyword);

    $this->view('mahasiswa/index', [
        'mahasiswa' => $data,
        'keyword' => $keyword
    ]);
}

```

Controller mengambil keyword pencarian dari URL, kemudian meminta data kepada Repository.

Data yang diperoleh kemudian dikirimkan ke View mahasiswa/index.

Gambar 10. Method index pada MahasiswaController

```

12 public function index(): void
13 {
14     $keyword = $_GET['search'] ?? '';
15
16     $data = $this->repository->all($keyword);
17
18     $this->view('mahasiswa/index', [
19         'mahasiswa' => $data,
20         'keyword' => $keyword
21     ]);
22 }
23

```

11. Proses Tambah Data Mahasiswa

Method store() digunakan untuk menerima data dari form dan meneruskannya kepada Repository.

Data yang diterima dari form meliputi:

```

$data = [
    'nim' => $_POST['nim'] ?? '',
    'nama' => $_POST['nama'] ?? '',

```

```
'email' => $_POST['email'] ?? ",
'prodi_id' => $_POST['prodi_id'] ?? ",
'angkatan' => $_POST['angkatan'] ?? "
];
```

Kemudian data disimpan melalui:

```
$this->repository->create($data);
```

Setelah proses selesai, pengguna diarahkan kembali ke halaman mahasiswa.

Gambar 11. Method store pada MahasiswaController

```
29 public function store(): void
30 {
31     $data = [
32         'nim' => $_POST['nim'] ?? '',
33         'nama' => $_POST['nama'] ?? '',
34         'email' => $_POST['email'] ?? '',
35         'prodi_id' => $_POST['prodi_id'] ?? '',
36         'angkatan' => $_POST['angkatan'] ?? ''
37     ];
38
39     $this->repository->create($data);
40
41     $this->redirect('/acara9/public/index.php?route=mahasiswa')
42 }
```

12. Proses Edit dan Update Data

Method edit() digunakan untuk mengambil data mahasiswa berdasarkan ID.

```
$id = (int) ($_GET['id'] ?? 0);
```

```
$mahasiswa = $this->repository->find($id);
```

Data tersebut kemudian dikirimkan ke halaman edit.

Untuk menyimpan perubahan digunakan method update():

```
$this->repository->update($id, $data);
```

Dengan demikian, Controller tetap hanya mengatur alur proses dan menyerahkan pengolahan database kepada Repository.

Gambar 12. Method edit dan update pada MahasiswaController

```

44 public function edit(): void
45 {
46     $id = (int) ($_GET['id'] ?? 0);
47
48     $mahasiswa = $this->repository->find($id);
49
50     if (!$mahasiswa) {
51         die('Data mahasiswa tidak ditemukan.');
```

```

59 public function update(): void
60 {
61     $id = (int) ($_POST['id'] ?? 0);
62
63     $data = [
64         'nim' => $_POST['nim'] ?? '',
65         'nama' => $_POST['nama'] ?? '',
66         'email' => $_POST['email'] ?? '',
67         'prodi_id' => $_POST['prodi_id'] ?? '',
68         'angkatan' => $_POST['angkatan'] ?? ''
69     ];
70
71     $this->repository->update($id, $data);
72
73     $this->redirect('/acara9/public/index.php?route=mahasiswa')
74 }
75
```

13. Proses Hapus Data

Method delete() digunakan untuk menghapus data mahasiswa berdasarkan ID.

```

public function delete(): void
{
    $id = (int) ($_GET['id'] ?? 0);

    $this->repository->delete($id);

    $this->redirect(
```

```
        '/acara9/public/index.php?route=mahasiswa'  
    );  
}
```

Proses penghapusan dilakukan dengan memanggil method delete() pada Repository.

Gambar 13. Method delete pada MahasiswaController

```
76     public function delete(): void  
77     {  
78         $id = (int) ($_GET['id'] ?? 0);  
79  
80         $this->repository->delete($id);  
81  
82         $this->redirect('/acara9/public/index.php?route=mahasiswa')  
83     }  
84 }
```

14. Tampilan Data Mahasiswa

Setelah implementasi OOP MVC dilakukan, halaman mahasiswa digunakan untuk menampilkan data yang diperoleh dari Repository.

Halaman tersebut menampilkan beberapa informasi, yaitu:

- NIM
- Nama
- Email
- Program Studi
- Angkatan
- Aksi Edit
- Aksi Hapus

Terdapat juga fitur pencarian berdasarkan NIM atau nama.

Gambar 14. Tampilan halaman data mahasiswa

Data Mahasiswa

[+ Tambah Mahasiswa](#)[Cari](#) [Reset](#)

No	NIM	Nama	Email	Prodi	Angkatan	Aksi	
1	2401002	Ani Wijaya	ani@email.com	Teknik Informatika	2024	Edit	Hapus
2	2401004	Sheryn Febrylia	e41250604@student.polije.ac.id	Teknik Informatika	2025	Edit	Hapus
3	2401007	Nila Agustin	2401007@student.polije.ac.id	Teknik Informatika	2026	Edit	Hapus
4	2401099	Mahasiswa API	2401099@student.polije.ac.id	Teknik Informatika	2026	Edit	Hapus

15. Fitur Pencarian Mahasiswa

Repository menyediakan pencarian berdasarkan NIM maupun nama.

Query yang digunakan:

WHERE m.nim LIKE :nim

OR m.nama LIKE :nama

Keyword pencarian kemudian digunakan sebagai nilai parameter:

\$search = '%' . \$keyword . '%';

```
$stmt->execute([  
    'nim' => $search,  
    'nama' => $search  
]);
```

Dengan fitur tersebut, pengguna dapat mencari data mahasiswa berdasarkan NIM atau nama.

Gambar 15. Hasil pencarian data mahasiswa

Data Mahasiswa

[+ Tambah Mahasiswa](#)[Cari](#) [Reset](#)

No	NIM	Nama	Email	Prodi	Angkatan	Aksi	
1	2401002	Ani Wijaya	ani@email.com	Teknik Informatika	2024	Edit	Hapus

16. Pengujian Tambah Data

Pengujian dilakukan dengan membuka form tambah mahasiswa dan memasukkan data mahasiswa baru.

Gambar 16. Form tambah data mahasiswa

Screenshot hasil: tampilkan halaman form tambah mahasiswa sebelum data disimpan.

Setelah data disimpan, data mahasiswa baru ditampilkan pada halaman daftar mahasiswa.

Gambar 17. Data mahasiswa setelah berhasil ditambahkan

Data Mahasiswa [+ Tambah Mahasiswa](#)

Cari berdasarkan NIM atau nama... [Cari](#) [Reset](#)

No	NIM	Nama	Email	Prodi	Angkatan	Aksi
1	23435465	Dinda	134356@student.polije.ac	Sistem Informasi	2025	Edit Hapus
2	2401002	Ani Wijaya	ani@email.com	Teknik Informatika	2024	Edit Hapus
3	2401004	Sheryn Febrylia	e41250604@student.polije.ac.id	Teknik Informatika	2025	Edit Hapus
4	2401007	Nila Agustin	2401007@student.polije.ac.id	Teknik Informatika	2026	Edit Hapus
5	2401009	Febiola	e41250609@student.polije.ac.id	Sistem Informasi	2025	Edit Hapus
6	2401099	Mahasiswa API	2401099@student.polije.ac.id	Teknik Informatika	2026	Edit Hapus

17. Pengujian Edit Data

Pengujian edit dilakukan dengan memilih tombol Edit pada salah satu data mahasiswa.

Setelah data diperbarui dan disimpan, data pada halaman mahasiswa berubah sesuai dengan nilai yang baru.

Gambar 18. Form edit data mahasiswa

Screenshot hasil: tampilkan form edit dengan data mahasiswa yang akan diubah.

Gambar 19. Hasil perubahan data mahasiswa

Edit Data Mahasiswa

NIM
2401010
Nama
Dinda Rachmawati
Email
2401010@student.polije.ac
ID Prodi
2
Angkatan
2025
<button>Kembali</button> <button>Update</button>

Data Mahasiswa

[+ Tambah Mahasiswa](#)

Cari berdasarkan NIM atau nama...

[Cari](#) [Reset](#)

No	NIM	Nama	Email	Prodi	Angkatan	Aksi
1	2401002	Ani Wijaya	ani@email.com	Teknik Informatika	2024	Edit Hapus
2	2401004	Sheryn Febrylia	e41250604@student.polije.ac.id	Teknik Informatika	2025	Edit Hapus
3	2401007	Nila Agustin	2401007@student.polije.ac.id	Teknik Informatika	2026	Edit Hapus
4	2401010	Dinda Rachmawati	2401010@student.polije.ac	Sistem Informasi	2025	Edit Hapus
5	2401099	Mahasiswa API	2401099@student.polije.ac.id	Teknik Informatika	2026	Edit Hapus

18. Pengujian Hapus Data

Pengujian hapus dilakukan dengan memilih tombol Hapus pada data mahasiswa.

Setelah proses selesai, data yang dipilih tidak lagi terdapat pada daftar mahasiswa.

Gambar 20. Hasil penghapusan data mahasiswa

Data Mahasiswa

Cari berdasarkan NIM atau nama...

localhost says

Yakin ingin menghapus data ini?

OK

Cancel

+ Tambah Mahasiswa

Cari

Reset

No	NIM	Nama	Email	Prodi	Angkatan	Aksi
1	23435465	Dinda	134356@student.polije.ac	Sistem Informasi	2025	Edit Hapus
2	2401002	Ani Wijaya	ani@email.com	Teknik Informatika	2024	Edit Hapus
3	2401004	Sheryn Febrylia	e41250604@student.polije.ac.id	Teknik Informatika	2025	Edit Hapus
4	2401007	Nila Agustin	2401007@student.polije.ac.id	Teknik Informatika	2026	Edit Hapus
5	2401009	Febiola	e41250609@student.polije.ac.id	Sistem Informasi	2025	Edit Hapus
6	2401099	Mahasiswa API	2401099@student.polije.ac.id	Teknik Informatika	2026	Edit Hapus

Data Mahasiswa

Cari berdasarkan NIM atau nama...

+ Tambah Mahasiswa

Cari

Reset

No	NIM	Nama	Email	Prodi	Angkatan	Aksi
1	23435465	Dinda	134356@student.polije.ac	Sistem Informasi	2025	Edit Hapus
2	2401002	Ani Wijaya	ani@email.com	Teknik Informatika	2024	Edit Hapus
3	2401004	Sheryn Febrylia	e41250604@student.polije.ac.id	Teknik Informatika	2025	Edit Hapus
4	2401007	Nila Agustin	2401007@student.polije.ac.id	Teknik Informatika	2026	Edit Hapus
5	2401099	Mahasiswa API	2401099@student.polije.ac.id	Teknik Informatika	2026	Edit Hapus

Pembahasan

Pada Acara 9 dilakukan refactoring struktur aplikasi dengan menerapkan konsep OOP MVC lanjutan.

Class Database bertanggung jawab terhadap koneksi database. MahasiswaRepository bertanggung jawab terhadap proses pengolahan data mahasiswa pada database. MahasiswaController bertanggung jawab mengatur alur request dan menggunakan Repository untuk mendapatkan atau mengubah data.

Hubungan antarobject dapat digambarkan sebagai berikut:

MahasiswaController



MahasiswaRepository



Database



PDO



MySQL

Struktur tersebut merupakan penerapan Object Composition, yaitu sebuah object menggunakan object lain untuk menjalankan tugas tertentu.

Dependency Injection diterapkan dengan memberikan MahasiswaRepository kepada MahasiswaController melalui constructor. Pada Repository, object Database juga diberikan melalui constructor.

Pada class Mahasiswa, seluruh atribut dibuat private. Akses terhadap atribut dilakukan menggunakan getter dan setter. Setter juga digunakan untuk melakukan validasi, seperti memastikan NIM berupa angka dan nama tidak boleh kosong.

Penerapan konsep tersebut membuat kode menjadi lebih terstruktur karena tanggung jawab setiap class dipisahkan. Hal ini sesuai dengan tujuan Acara 9 yang menekankan penerapan OOP dalam pengembangan backend web.

Kesimpulan

Berdasarkan praktikum Acara 9, dapat disimpulkan bahwa konsep OOP dapat diterapkan untuk membuat aplikasi web yang lebih terstruktur dan mudah dikelola.

Object Composition diterapkan melalui hubungan antara Controller, Repository, dan Database. Dependency Injection diterapkan dengan memberikan dependency melalui constructor sehingga Controller tidak perlu membuat dependency secara langsung. Getter dan Setter digunakan untuk mengatur akses terhadap atribut private pada class Mahasiswa.

Selain itu, setter dapat digunakan untuk melakukan validasi data sebelum data tersebut digunakan oleh aplikasi. Repository juga berhasil digunakan sebagai tempat pengelolaan operasi CRUD data mahasiswa. Dengan penerapan konsep tersebut, aplikasi dapat mempertahankan fungsi utama seperti menampilkan, mencari, menambah, mengubah, dan menghapus data mahasiswa, sekaligus memiliki struktur kode yang lebih terorganisasi.